LIFELINE: Algorithm Selection Report (Proposal)

Group 66 | Section AI-A | FAST-NUCES Islamabad | BS Artificial Intelligence | Irtaza Shahid (24I-0104), Muhammad Harris (24I-0020)

Module	Problem (Variables)	Candidates & Complexities	Selection	Justification & Failure Case
A. Relief Inventory	Max sum(v_i) s.t. sum(w_i) <= W, indivisible crates. Vars: N crates, capacity W.	1. 0/1 Knapsack DP: O(N*W) 2. Branch & Bound: O(2^N) 3. Greedy density: O(N log N)	**Primary:** 0/1 DP **Baseline:** Greedy value/kg	DP guarantees global optimum. Greedy fails severely on adversarial sets (Case 47 achieves <= 60% of optimum).
B. Route Planning	Find cheapest path u->v with edge mins + target penalty, plus reachable set. Vars: V locs, E roads.	1. Dijkstra + Min-Heap: O((V+E) log V) 2. Bellman-Ford: O(V*E) 3. Breadth-First Search: O(V+E)	**Primary:** Dijkstra (Heap) **Baseline:** BFS (fewest roads)	Non-negative costs make Dijkstra optimal. BFS only minimizes hops and incurs severe cost inflation (Case 48 costs >= 1.5x).
C. Rescue Scheduling	Select max compatible windows (count or value), touching allowed. Vars: M windows in [0, 1440].	1. Weighted DP + BinSearch: O(M log M) 2. Greedy Earliest Finish: O(M log M) 3. Brute Force: O(2^M)	**Primary:** Weighted DP **Baseline:** Greedy Earliest Finish	Weighted DP finds optimal values under heavy overlap. Greedy only optimizes unweighted count and fails on value maximization.
D. Emergency Queue	Release 1 alarm per tick in urgency order (priority 10 down to 1). Vars: A alarms, T ticks.	1. Binary Min-Heap: O(A log A) 2. Fibonacci Heap: O(A + A log A) 3. Unsorted List: O(A^2)	**Primary:** Binary Max-Heap **Baseline:** Unsorted Array Scan	Heap guarantees O(log A) push/pop, sustaining bursts up to 1.8 alarms/tick (Urban Flood). Unsorted scans cause starvation.
E. Radio Logs	Pattern search (overlaps); min edit script; prefix-free code. Vars: N log len, P pat len, L reports.	1. KMP: O(N+P) vs Naive O(N*P) 2. Wagner-Fischer: O(L1*L2) 3. Huffman Coding: O(S log S)	**Primary:** KMP + Wagner-Fischer + Huffman **Baseline:** Naive search	KMP eliminates backtracking on near-miss logs (Case 49 triggers O(N*P) in naive search). Huffman minimizes transmission bits.
F. Network Planning	Connect all reachable nodes with minimum total cable cost. Vars: V locs, E roads.	1. Kruskal + DSU: O(E log V) 2. Prim + Heap: O(E + V log V) 3. BFS Spanning Tree: O(V+E)	**Primary:** Kruskal with DSU **Baseline:** BFS Spanning Tree	Sparse-to-medium networks (E <= 3V) favor Kruskal with path compression. BFS tree ignores cable weights and yields suboptimal cost.
G. Chains & Cuts	Optimal matrix chain parenthesization; max revenue drum cut. Vars: n stages, drum length L.	1. Matrix Chain DP: O(n^3) 2. Drum Cutting DP: O(L^2) 3. Greedy price/meter cut: O(L log L)	**Primary:** 1D & 2D DP **Baseline:** Greedy Density Cutting	Non-monotonic drum prices cause greedy cutting to fail. Matrix chain DP finds exact global minimum scalar multiplications.
H. Hard Planning	Solve 3-SAT, Subset Sum, closed TSP, and Vertex Cover. Vars: N vars/items, V nodes, k cover.	1. DPLL Backtracking: O(2^N) 2. Subset Sum DP: O(N*Sum) 3. TSP Held-Karp / Branch & Bound 4. Vertex Cover FPT: O(2^k * V)	**Primary:** DPLL + Exact DP / B&B **Baseline:** Random Walk / 2-Approx	Exact DP/B&B verifies planted solutions and handles phase-transition clause densities (ratio 3.5 to 4.8). Approximations fail exact targets.
I. Nearest & Checksums	Pair of sites at min squared dist; product of serial numbers. Vars: n sites, D digits.	1. Closest Pair D&C: O(n log n) 2. Karatsuba Mult: O(D^1.585) 3. All-pairs O(n^2), Grade-School O(D^2)	**Primary:** Divide & Conquer + Karatsuba **Baseline:** O(n^2) Brute Force	Divide-and-conquer scales efficiently to Tier L (n=500 sites). Karatsuba speeds up multi-precision arithmetic on serial products.

LIFELINE: Iteration 1 Work Division & Iteration 2 Plan

Group 66 | Section AI-A | BS Artificial Intelligence | Equal 50/50 Team Collaboration (2 Members)

Member	Iteration 1 Contribution & Iteration 2 Proposed Decision Modules
Member 1: Irtaza Shahid **Name: Irtaza Shahid** **Roll No: 24I-0104** **Section: AI-A** **Degree: BS Artificial Intelligence** **Role: Core Systems, Graph Algorithms & Traps Architect**	**Iteration 1 Contributions:** • **Generator Core:** Architected `main.py`, CLI arguments, workbook serializer for all 20 sheets, and natural sorting logic. • **Data Modules Built:** Crates generator (weight, value, expiry, serials), Locations (penalties, depot/shelter assignments), Roads network generator, and Wave Closures. • **Hostile Traps:** Implemented Case 46 (Dock-Order monotone crate list) and Case 48 (Shortest-Looks-Cheapest 600-node chain with 7.33x BFS cost penalty). • **Validator Engineering:** Implemented checks V1 (Sheet schema and consecutive case IDs), V2 (Numerical ranges and capacity thresholds), V3 (Foreign key and reference validation), V4 (Network connectivity via BFS and wave disjointness), and V7 (Delta Flood, Mountain, and Coastal scenario signatures). • **Documentation:** Authored Approach Note Part 1 (Story to numbers) and Part 3 (Traps specification and adversarial mechanics). **Proposed Iteration 2 Decision Modules:** • **Module A (Relief Inventory):** 0/1 Knapsack Dynamic Programming with space-optimized state array. • **Module B (Route Planning):** Dijkstra's Algorithm with binary min-heap supporting dynamic road closures and target penalties. • **Module D (Emergency Queue):** Event-driven Priority Queue engine managing incident streams under high alarm rates. • **Module F (Network Planning):** Kruskal's Minimum Spanning Tree with Disjoint Set Union (path compression and union-by-rank).
Member 2: Muhammad Harris **Name: Muhammad Harris** **Roll No: 24I-0020** **Section: AI-A** **Degree: BS Artificial Intelligence** **Role: Planted Objects, String Analysis & Hard Problems Lead**	**Iteration 1 Contributions:** • **Data Modules Built:** Radio Log & Chunking, Patterns (2 to 40 chars), Reports & Levenshtein mutation, Rescue Windows (touching pairs), Incidents & Poisson ticks, Sites & Planted TSP Tour, Pipeline Dimensions, Drum Prices, Sensor Cover, Safety Rules (3-SAT), and Rations (Subset Sum). • **Exact Tier S Solvers:** Implemented exact algorithms for `claims` sheet: 0/1 Knapsack DP, Earliest Finish Time Greedy, Weighted Interval DP, Rod Cutting DP, Exact TSP DP, and Minimum Vertex Cover bitmask search. • **Hostile Traps:** Implemented Case 47 (Best-Looking-Crate greedy trap <= 60%) and Case 49 (Near-Miss 60,000-char string matching worst-case). • **Validator Engineering:** Implemented checks V5 (Pattern counts, edit distance bounds, and Tier S claims recomputations), V6 (Planted tour closed Manhattan length, sensor cover, 3-SAT satisfying assignment, and rations sum), and V8 (Trap rules 46-49 verification). Built `create_broken_tests.py` (10 corrupted cases) and `run_suite.py`. • **Documentation:** Authored Approach Note Part 2 (Planted objects proof) and Part 4 (What will be hard), and Proposal Page 1 Table. **Proposed Iteration 2 Decision Modules:** • **Module C (Rescue Scheduling):** Weighted Interval Scheduling DP with binary search for predecessor compatible windows. • **Module E (Radio Logs):** Knuth-Morris-Pratt (KMP) string matching, Wagner-Fischer edit scripts, and Huffman compression. • **Module G (Chains & Cuts):** Matrix Chain Multiplication DP parenthesization and Unbounded Rod Cutting DP. • **Module H (Hard Planning):** DPLL SAT solver with unit propagation, Subset Sum meet-in-the-middle, and Vertex Cover FPT. • **Module I (Nearest & Checksums):** Divide-and-Conquer Closest Pair of Points and Karatsuba Fast Multiplication.